

Flag : `picoCTF{L00k5_l1k3_y0u_solv3d_it_...}`

📄 Résumé de la faille

Le formulaire de login insère directement ce que l'utilisateur tape dans une requête SQL, sans le nettoyer. En injectant `admin' --` dans le champ username, on **commente la vérification du mot de passe** et on se connecte en tant qu'admin **sans connaître son mot de passe**. Le flag était ensuite dans le code source de la page (Ctrl+U).

Faille : injection SQL → contournement d'authentification (authentication bypass).

🔑 Le concept clé (le déclic)

On ne réécrit PAS la requête. On injecte juste un bout dans celle du serveur.

Le serveur a déjà son code figé :

```
SELECT * FROM users WHERE name = '[MON_INPUT]' AND password = '[...]'
```

`[MON_INPUT]` = exactement ce que je tape dans le champ username. Je ne peux pas ajouter de SELECT/FROM (ils existent déjà). Je peux seulement manipuler ce qui est entre les guillemets.

🔧 Méthodo — les étapes

- 1. Tester si c'est injectable** - Taper un guillemet simple `'` dans le champ username. - Le comportement du site change (le message "Login failed" disparaît) → la saisie casse la requête → **c'est vulnérable**.
- 2. Comprendre la requête cible** - Objectif : que la requête finale soit juste `SELECT * FROM users WHERE name = 'admin'` et que la partie `AND password = '...'` soit **ignorée**.
- 3. Construire l'injection (3 pièces)** - `admin` → je dis que je suis admin - `'` → je **ferme** le guillemet ouvert par le serveur - `--` → symbole de **commentaire SQL** : tout ce qui suit est ignoré (attention : `--` demande souvent un ESPACE après en SQLite/MySQL)
- 4. L'injection finale**

```
Champ username : admin' --
Champ password : (vide, ignore)
```

Requête resultante côté serveur :

```
SELECT * FROM users WHERE name='admin' --' AND password=''
```

Le `--` commente `' AND password=''` → il ne reste que `name='admin'` → connecté sans mot de passe. -> "Logged in!"

- 5. Trouver le flag** - Message : "the flag is in plain sight" - Reflexe : **Ctrl+U** (code source) → flag présent dans le HTML.

🔑 Les briques SQLi à retenir

Brique	Rôle
<code>'</code>	Fermer/casser la chaîne de caractères du serveur
<code>--</code> (ou <code>#</code>)	Commenter = ignorer la fin de la requête
<code>OR 1=1</code>	Rendre une condition toujours vraie
<code>admin' --</code>	Bypass classique : se logger comme admin sans password

Variantes si `admin' --` ne marche pas : - `admin' --` (avec espace après les tirets) - `admin'#` (le `#` commente dans certaines bases) - `admin'/*` (commentaire bloc)

✅ Réflexes à retenir

- **Tester l'injectabilité** : mettre un `'` et observer si le site réagit bizarrement (erreur, message qui change).
- **Ne PAS réécrire la requête** : on injecte un fragment dans celle du serveur, entre ses guillemets.
- **Commenter la fin** avec `--` pour neutraliser la vérif du password.
- **Toujours regarder le code source** (Ctrl+U) : le flag y est souvent "in plain sight".

🔧 Dans la vraie vie

L'injection SQL est une des failles web les plus critiques et les plus anciennes. Elle permet, selon les cas : - de contourner un login (comme ici) - d'extraire toute une base de données (mots de passe, données perso) - parfois d'aller jusqu'à l'exécution de commandes sur le serveur

Protection côté dev : requêtes préparées (prepared statements / requêtes paramétrées), qui séparent le code SQL des données saisies par l'utilisateur -> l'input ne peut plus modifier la structure de la requête. Ne JAMAIS concatener directement l'input dans une requête SQL.

Outil d'automatisation : **sqlmap** (détecte et exploite les SQLi automatiquement) — à découvrir sur des challenges plus avancés.